

getpriority()

Tema 4

Diogo Pontes Vieira Costa
Guilherme Silva Cotrim
Felipe Almeida Cardoso
Matheus Vieira de Assis
Pedro Paulo Veri Marquez

Universidade Federal de Uberlândia

2026 - 1

Sumário

- 1 Tema Escolhido
- 2 Motivação do Projeto
- 3 Algoritmos e Mecanismos Implementados
- 4 Funcionamento da Solução
- 5 Proposta de Implementação no XV6
- 6 Encerramento

Tema Escolhido

O tema escolhido para este projeto é a implementação de um mecanismo de prioridade de processos no sistema operacional XV6, incluindo as chamadas de sistema `getpriority` (foco principal) e `setpriority` (importante para um programa de testes mais completo), além de uma proposta de integração com o escalonador.

Motivação do Projeto

O XV6 é um sistema operacional educacional minimalista que implementa um escalonador simples baseado em round-robin, onde todos os processos têm a mesma prioridade. Apesar de ser suficiente para fins didáticos, esse modelo não representa sistemas reais, que utilizam políticas de escalonamento mais sofisticadas, frequentemente baseadas em prioridade. A motivação deste projeto é justamente aproximar o XV6 de um cenário mais realista, permitindo diferenciar processos com base em prioridade, controlar o uso da CPU e explorar conceitos fundamentais de sistemas operacionais, como escalonamento e concorrência. O objetivo dessa abordagem é evitar starvation, garantindo que todos os processos eventualmente recebam tempo de CPU, mesmo aqueles com prioridade inicial baixa. Além disso, essa modificação permite estudar, na prática, como o kernel gerencia processos.

Algoritmos e Mecanismos Implementados

- ❶ **Armazenamento de Prioridade:** Cada processo passa a ter um novo campo '*priority*' na estrutura *proc*, representando sua prioridade.
- ❷ **Chamadas de Sistema:** Foram definidas duas *syscall*:
 - *getpriority(int pid)*: retorna a prioridade de um processo
 - *setpriority(int pid, int prio)*: altera a prioridade de um processo

Essas chamadas envolvem:

- Captura de argumentos com *argint()*
 - Busca do processo na tabela global (*proc[]*)
 - Uso de *locks* (*p*→*lock*) para garantir consistência
- ❸ **Política de escalonamento (proposta):** Além do uso de prioridade, a solução também propõe o uso de *aging*, que consiste em aumentar gradualmente a prioridade de processos que permanecem muito tempo na fila de prontos. A cada ciclo do escalonador, processos que não foram executados podem ter sua prioridade incrementada, até um limite máximo. Isso garante que processos de baixa prioridade eventualmente sejam executados, evitando *starvation*.

Funcionamento da Solução

O funcionamento geral ocorre da seguinte forma:

- 1 Um processo pode consultar sua prioridade ou a de outro processo usando `getpriority`
- 2 Um processo pode alterar prioridades via `setpriority`
- 3 O kernel armazena essa informação na estrutura de processos
- 4 O escalonador pode utilizar essa prioridade para decidir qual processo executar

Por exemplo: um processo com prioridade maior tende a ser executado antes, processos com prioridade menor podem esperar mais tempo. Isso introduz um comportamento mais próximo de sistemas reais, onde tarefas críticas recebem mais CPU.

Proposta de Implementação no XV6

A implementação no XV6 será dividida em etapas:

Etapas 1: Estrutura de Dados

- Adicionar o campo *priority* em *struct proc*

Etapas 2: *Syscalls*

- Modelar *sys_getpriority* e *sys_setpriority*
- Registrar em *syscall.c*
- Criar *stubs* em *user/usys.pl*

Etapas 3: Integração com o escalonador

- Modificar o *scheduler()* para considerar prioridade

Etapas 4: Validação

- Testar com múltiplos processos
- Verificar se prioridades influenciam execução
- Avaliar o impacto do mecanismo de *aging* no tempo de espera dos processos

Nesta solução, o mecanismo de *aging* é incorporado ao escalonador com o objetivo de evitar *starvation*. Processos que permanecem por muito tempo na fila de prontos têm sua prioridade aumentada gradualmente, até um limite máximo, garantindo que eventualmente sejam escalonados. Por fim, essas características aproximam ainda mais o sistema dos kernels reais.

Encerramento

Em resumo, este projeto demonstra como um sistema operacional simples pode ser estendido para suportar políticas mais avançadas de escalonamento, permitindo uma compreensão mais profunda do funcionamento interno de kernels modernos.